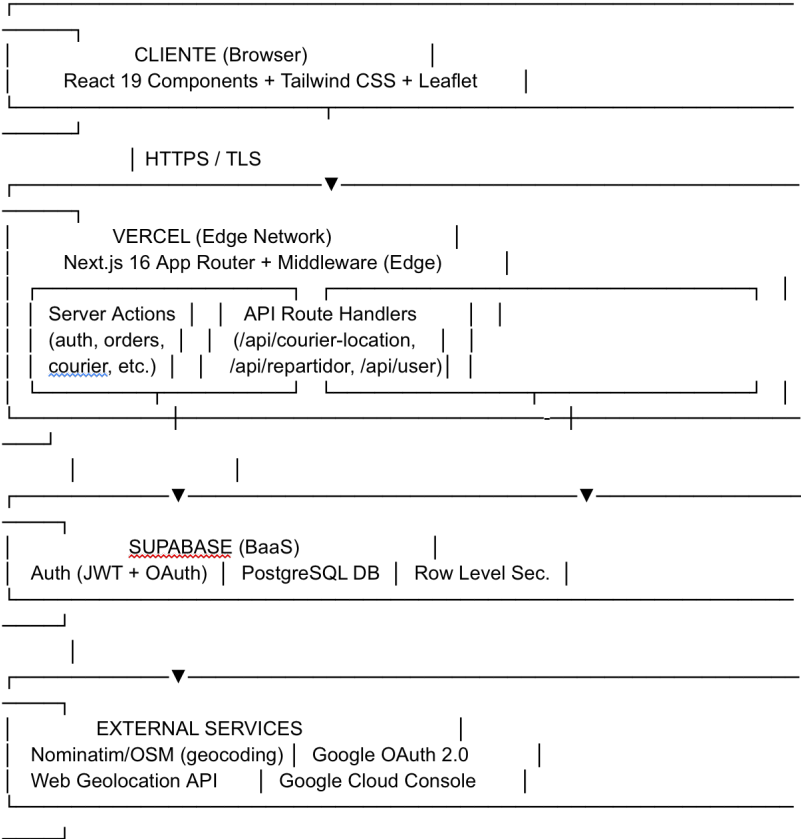


Arquitectura Tecnológica — BiciSend

1. Visión General del Sistema

BiciSend es una **aplicación web monolítica con separación de responsabilidades por capas**, construida sobre Next.js 16 con App Router. No tiene un backend separado: toda la lógica del servidor vive dentro del mismo proyecto mediante **Server Actions** y **Route Handlers**, comunicándose con Supabase como capa de datos y autenticación.

text



2. Estructura de Directorios (Real del Repositorio)

Basado en la inspección directa del repositorio :

text

```
bicisend-webapp/  
├── middleware.ts # Route protection (Edge Runtime)
```

```

├── next.config.ts          # Next.js configuration
├── tailwind.config.ts     # Design system tokens
├── tsconfig.json          # TypeScript strict mode
├── public/                # Static assets
├── src/
│   ├── app/
│   │   ├── layout.tsx     # Root layout (fonts, metadata)
│   │   └── page.tsx       # Landing page (public)
│   │   ├── (auth)/       # Auth route group (no layout shared)
│   │   │   ├── login/
│   │   │   └── register/
│   │   ├── (protected)/  # Protected route group
│   │   │   ├── cliente/
│   │   │   │   ├── layout.tsx # Client shell + navbar
│   │   │   │   ├── page.tsx  # Client dashboard
│   │   │   │   ├── nuevo/    # New order creation
│   │   │   │   └── envios/    # Order history
│   │   │   ├── repartidor/
│   │   │   │   ├── layout.tsx # Courier shell + navbar
│   │   │   │   ├── page.tsx  # Courier dashboard
│   │   │   │   ├── _components/ # Co-located private components
│   │   │   │   ├── activos/  # Active deliveries
│   │   │   │   └── ganancias/ # Earnings history
│   │   ├── actions/       # Next.js Server Actions
│   │   │   ├── auth.ts     # Login, register, logout
│   │   │   ├── orders.ts   # Create, update orders
│   │   │   ├── courier.ts  # Accept, complete deliveries
│   │   │   ├── courier-location.ts # GPS location management
│   │   │   └── profile.ts  # User profile updates
│   └── api/               # API Route Handlers

```

```

├── courier-location/      # GET latest courier GPS
├── repartidor/           # Courier-specific endpoints
├── user/                 # User profile endpoints
├── auth/                 # OAuth callback handler
├── callback/             # Google OAuth redirect target
├── components/           # Shared UI components
├── LeafletMap.tsx       # Interactive map renderer
├── LeafletAddressAutocomplete.tsx # OSM address search (iOS-fixed)
├── DeliveryTrackingMap.tsx # Client-side tracking map
├── CourierLocationTracker.tsx # GPS sender (courier device)
├── GoogleMapsNavigationButton.tsx # Deep link to Google Maps
├── MobileMenu.tsx      # Responsive nav menu
├── lib/
│   ├── supabase/
│   │   ├── client.ts     # Browser Supabase client
│   │   ├── server.ts     # Server-side Supabase client
│   │   └── middleware.ts  # Edge Supabase client + cookie sync
│   ├── utils/
│   │   └── geo-utils.ts  # Haversine, coord validation
├── types/
│   ├── database.types.ts # Auto-generated Supabase types
│   └── courier-location.ts # GPS payload types

```

3. Capas de la Arquitectura

3.1 Capa de Presentación (Frontend)

Toda la interfaz vive en `src/app/` y `src/components/` .

Tecnologías:

Tecnología	Versión	Rol
Next.js	16 (App Router)	Framework principal, SSR/SSG/RSC
React	19	UI components
TypeScript	Strict mode	Tipado seguro
Tailwind CSS	4.x	Utility-first styling

